

Stima della cardinalità nei flussi di dati distribuiti: merge e compatibilità semantica degli algoritmi di sketch

Seminario magistrale

Daniele Ferroli

Università degli Studi di Udine

A.A. 2024–2025

- ① Problema del conteggio degli elementi distinti
- ② Fondamenti teorici
- ③ Analisi dello stato dell'arte
- ④ Implementazione: architettura, hash, dataset e validazione
- ⑤ Risultati principali
- ⑥ Conclusioni, limiti e sviluppi futuri

Problema: contare il numero di elementi distinti

- I **sistemi** moderni ricevono aggiornamenti in **modo continuo** e su **enormi** moli di **dati**.
- Il **problema** è: **stimare** quanti **elementi distinti** sono comparsi in un flusso.
- **Vincolo**: non poter conservare l'intero flusso in memoria.
- **Applicazioni**: *monitoraggio di rete, telemetria, osservabilità, log, ottimizzazione di interrogazioni e sistemi distribuiti.*

Perché stimare?

Quando il volume dei dati cresce, il conteggio esatto diventa costoso in memoria e tempo, mentre la risposta deve arrivare con una bassa latenza.

$$S = \langle A, G, T, C, C, G, A \rangle$$

$$\text{elementi osservati} = 7 \quad \text{elementi distinti} = |\{A, C, G, T\}| = 4$$

- Le ripetizioni non aumentano il numero di elementi distinti.
- Il valore cercato non è la lunghezza del flusso, ma il numero di valori diversi comparsi.
- Nei casi reali il flusso può essere molto grande o non avere una dimensione nota a priori.

domanda: quanti valori diversi ho visto finora?

$$V \leftarrow V \cup \{x_i\} \quad \text{risposta} = |V|$$

Arriva	V
A	$\{A\}$
G	$\{A, G\}$
T	$\{A, G, T\}$
C	$\{A, C, G, T\}$
C, G, A	invariato

- Si mantiene l'insieme V dei valori già osservati.
- Ogni nuovo elemento aggiorna V , se non era già presente.
- La risposta è esatta: $|V| = 4$.

$$\text{memoria richiesta} \propto |V|$$

- La soluzione naive deve conservare un rappresentante per ogni elemento distinto.
- Se il numero di distinti cresce, cresce anche lo stato mantenuto in memoria.
- Nei flussi continui il valore finale può non essere noto in anticipo.
- Spesso serve aggiornare la risposta con latenza bassa, senza analizzare tutto a posteriori.

Motivazione

Il problema non è definire cosa contare: è contarlo con memoria limitata.

Idea dello sketch

$$x_i \longrightarrow \text{hash} \longrightarrow M \longrightarrow \hat{F}_0$$

$$M \leftarrow \text{Update}(M, x_i) \quad \hat{F}_0 \leftarrow \text{Query}(M)$$

- Lo sketch sostituisce l'insieme V con uno stato compatto M .
- A ogni elemento lo stato viene aggiornato, senza conservare tutto il flusso.
- La risposta diventa una stima $\hat{F}_0$, non necessariamente il valore esatto.
- L'hash rende osservabili proprietà statistiche del flusso.

Passaggio concettuale

Da ricordare quali elementi sono comparsi a mantenere abbastanza informazione per stimare quanti ne sono comparsi.

$$S_1 \longrightarrow K_1 \quad S_2 \longrightarrow K_2 \quad S_3 \longrightarrow K_3$$

$$\text{Query}(K_1 \oplus K_2 \oplus K_3) \longrightarrow \hat{F}_0(S_1 \cup S_2 \cup S_3)$$

- Il flusso può essere diviso tra nodi o partizioni temporali.
- Ogni nodo mantiene solo il proprio sketch locale K_i .
- Per stimare il numero globale di distinti, gli sketch locali devono essere uniti.
- Il merge dovrebbe comportarsi come se avessimo processato tutto il flusso in un solo sketch.

Punto critico

Il merge non combina solo stati: deve preservare la semantica con cui quegli stati sono stati costruiti.

- Quando il merge di due sketch è equivalente al processamento centralizzato?
- Quali condizioni devono coincidere: algoritmo, hash, seed, precisione?
- Cosa succede se cambia la semantica dell'hash?
- Cosa succede se cambia solo la precisione dello sketch?
- Quali mismatch sono recuperabili e quali rendono il risultato privo di significato?

Contributo

Classificare e misurare sperimentalmente i casi valid, recoverable e invalid.

$$S = \langle x_1, x_2, \dots, x_s \rangle, \quad x_i \in \mathcal{U}$$

$$f(a) = |\{i \mid x_i = a\}| \quad F_0 = |\{a \in \mathcal{U} \mid f(a) > 0\}|$$

- S è un flusso ordinato di elementi osservati.
- $f(a)$ conta quante volte compare la chiave a .
- F_0 è la cardinalità del supporto di f , cioè il numero di elementi distinti.
- Nel lavoro si considera il modello *insertion-only*: gli aggiornamenti aggiungono occorrenze.

$$M \leftarrow \text{Update}(M, x_i), \quad \widehat{F}_0 \leftarrow \text{Query}(M)$$

- Lo stato M deve essere piccolo rispetto alla dimensione del flusso.
- L'aggiornamento deve essere veloce, perché avviene per ogni elemento.
- L'accuratezza viene letta come scostamento tra $\widehat{F}_0$ e F_0 .
- L'obiettivo pratico è scambiare memoria per una stima controllata.

Garanzia teorica tipica

Una stima (ε, δ) -approssimata richiede $\Pr(|\widehat{F}_0 - F_0| \leq \varepsilon F_0) \geq 1 - \delta$.

Linea evolutiva degli sketch

Metodo	Stato mantenuto	Miglioramento introdotto
Probabilistic Counting	Una bitmap	Eventi rari dell'hash
PCSA	Più bitmap virtuali	Mediazione su sottostrutture
LogLog	Registri	Massimi di rango
HyperLogLog	Registri	Media armonica e correzioni
HyperLogLog++	Sparso/denso	Bias correction e gestione pratica dei regimi piccoli

Idea comune: l'hash trasforma gli elementi in segnali pseudo-casuali; lo sketch osserva eventi rari per inferire la scala di F_0 .

Probabilistic Counting

$$B = (b_1, \dots, b_L)$$

$$\rho(y) = \min\{1 + \nu_2(y), L\}$$

$$b_{\rho(h(x))} \leftarrow 1$$

- Lo stato è una bitmap inizializzata a zero.
- ρ misura quanti zeri finali compaiono nell'hash, più uno.
- Eventi con ρ alto sono rari: vederli suggerisce molti distinti.

$$R(B) = \min\{r \geq 1 : b_r = 0\}, \quad \widehat{F}_0 \approx \frac{2^{R(B)}}{\phi}$$

Limite

Una sola bitmap è molto compatta, ma produce varianza elevata.

$$m = 2^k, \quad h(x) = \underbrace{j}_{\text{registro}} \underbrace{w}_{\text{rango}}$$

$$M[j] \leftarrow \max\{M[j], \rho(w)\}$$

- Il prefisso dell'hash sceglie uno dei m registri.
- Il suffisso produce il rango $\rho(w)$.
- Ogni registro mantiene il massimo rango osservato.
- Se F_0 cresce, i massimi tendono a crescere come $\log_2(F_0/m)$.

$$A = \frac{1}{m} \sum_{j=0}^{m-1} M[j], \quad \widehat{F_0} = \alpha_m m 2^A$$

Da LogLog a HyperLogLog

Stato

$$M[j] \leftarrow \max\{M[j], \rho(w)\}$$

HLL conserva la stessa struttura a registri di LogLog.

Stimatore

$$Z = \sum_{j=0}^{m-1} 2^{-M[j]}, \quad E = \alpha_m \frac{m^2}{Z}$$

La media armonica riduce il peso dei registri estremi.

- Per cardinalità piccole usa *linear counting*: $m \log(m/V_0)$.
- Nel regime centrale usa la stima armonica E .
- Vicino al limite dello spazio hash applica una correzione di saturazione.

Da HyperLogLog a HyperLogLog++

- HLL++ mantiene la logica a registri, ma usa hash a 64 bit.
- Per cardinalità piccole usa una rappresentazione sparsa.
- Quando lo stato sparso non conviene più, passa alla rappresentazione densa.
- Corregge empiricamente il bias tramite tabelle dipendenti dalla precisione.
- Sceglie tra linear counting e stima corretta con soglie empiriche.

$$E_{\text{corr}} = \begin{cases} E - \beta(E, k), & E \leq 5m, \\ E, & \text{altrimenti.} \end{cases}$$

Ruolo nella tesi

HLL++ è il riferimento pratico principale per gli esperimenti di streaming e di merge.

Merge e compatibilità semantica

Bitmap

Per Probabilistic Counting e PCSA:

$$B = B_1 \vee B_2$$

Registri

Per LogLog, HLL e HLL++:

$$M[j] = \max\{M_1[j], M_2[j]\}$$

Condizione

Il merge coincide con il processamento centralizzato solo se lo stato ha la stessa semantica: algoritmo, hash, seed e parametri compatibili.

Classificazione dei casi di merge

valid	recoverable	invalid
Merge diretto. Stesso algoritmo, stessa hash, stesso seed, stessi parametri.	Merge corretto dopo normalizzazione. HLL++ con precisioni diverse e stessa semantica hash.	Merge privo di significato nel modello. Hash family o seed diversi.

Strategie operative: direct, reject, reduce_then_merge; unsafe_naive_merge è solo un controllo negativo.

Algoritmi implementati nel framework

ID	Metodo	Parametro
naive	NaiveCounting	riferimento interno
pc	Probabilistic Counting	L bitmap
ll	LogLog	$k, m = 2^k$
hll	HyperLogLog	$k, L = 32$
hllpp	HyperLogLog++	p , hash 64 bit

- Interfaccia comune: `process`, `count`, `merge`, `reset`.
- La funzione di hash viene iniettata nel costruttore.
- Il merge è parte del contratto, non una funzione esterna.
- `naive` serve come riferimento esatto nei test.

Obiettivi progettuali dell'implementazione

- Uniformare algoritmi diversi sotto un contratto comune.
- Separare la responsabilità dello sketch dalla responsabilità dell'hashing.
- Rendere tracciabili seed, parametri, dataset, modalità e risultati.
- Tenere separati aggiornamento dello sketch, checkpoint, metriche e serializzazione.

Principio guida

Il progetto non è solo una raccolta di algoritmi: è una piattaforma di confronto riproducibile.

Architettura generale del sistema

Algoritmi

Sketch concreti e catalogo degli identificatori: h11pp, h11, 11, pc.

Supporto

Dataset binario, lettura per partizione, funzioni di hash e metadati di esecuzione.

Valutazione

CLI, job di esperimento, framework di simulazione, metriche e scrittura dei risultati CSV.

Scelta progettuale

Algoritmi, hash, dataset e metriche sono separati per rendere gli esperimenti riproducibili ed estendibili.

Stato

- Hash sempre a 64 bit.
- Rappresentazione sparsa con `tmpSet` e `sparseList`.
- Rappresentazione densa con vettore di registri
- Conversione quando lo sparso non conviene più in memoria.

Stima e merge

- Tabelle di bias e soglie in `hllpp_tables.cpp`.
- Merge diretto solo a parità di precisione.
- `reducedTo(...)` per normalizzare alla precisione minima.
- Base del caso recoverable.

Hash, riproducibilità e scelte progettuali

Livello hash

- Interfaccia comune: `hash64`, `hash32`, `name`.
- Funzioni supportate: `splitmix64`, `xxhash64`, `murmurhash3`, `siphash24`.
- L'hash è parte della semantica dello sketch, non un dettaglio di implementazione.

Riproducibilità

- Il seed del dataset è nell'header binario.
- Nel merge eterogeneo i due lati possono avere seed/hash separati.
- I metadati del dataset sono derivati automaticamente.

Framework sperimentale

Modalità

- `evaluateStreaming`: prefissi e checkpoint.
- `evaluateMergePairs`: coppie omogenee.
- `evaluateHeterogeneousMergePairs`: contesti locali distinti.

Output

- Risultati prodotti in memoria.
- Scrittura separata tramite `CsvResultWriter`.
- Percorsi con namespace, modalità, algoritmo, hash e parametri.
- Nel merge eterogeneo: campi `left_*` e `right_*`.

Il framework deriva `runs`, `sample_size` e `seed` dai metadati del dataset.

Dataset prefix_constant_rho e validazione

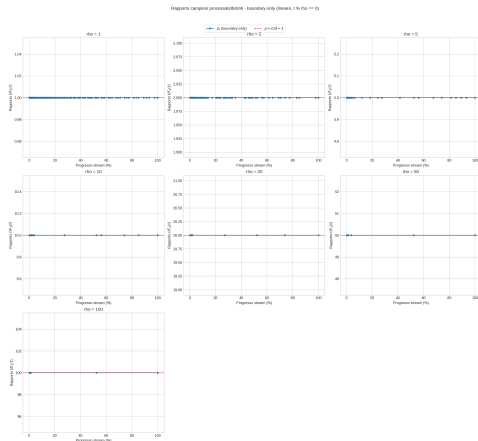
- Dataset sintetico controllato con $n = 10^7$, 50 partizioni, seed = 21041998.
- Rapporto controllato:

$$\rho = \frac{n}{d} \in \{100, 50, 20, 10, 5, 2, 1\}.$$

- Una nuova prima occorrenza per ogni blocco di lunghezza ρ :

$$F_0(j \cdot \rho) = j.$$

- Il bitset di verità permette di ricostruire $F_0(t)$ sui prefissi.



La validazione empirica conferma il comportamento atteso di $\rho_t = t/F_0(t)$ ai bordi di blocco.

Test dedicati coprono algoritmi, merge, framework, schema CSV e generatore del dataset.

Disegno sperimentale

- Ambiente di test: AMD Ryzen 7 9700X, 32 GB DDR5, Clang 22.1.1, CMake 4.3.0.
- Tre modalità di valutazione:
 - **streaming**: andamento della stima lungo i prefissi;
 - **merge omogeneo**: confronto tra merge e riferimento seriale;
 - **merge eterogeneo**: casi *valid*, *recoverable*, *invalid*.
- Due viste complementari in streaming:
 - caso A: stima in funzione della cardinalità reale del prefisso;
 - caso B: stima in funzione di $\rho_t = t/F_0(t)$, con

$$F_0^* \in \{1000, 2000, 5000, 10000, 20000, 50000, 100000\}.$$

- Metriche: stima media, dispersione, errore relativo medio ai punti finali.
- Gli esperimenti di merge sono concentrati su HyperLogLog++.

Streaming

- Dataset:

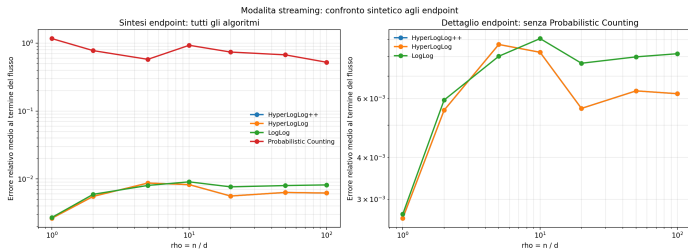
$$d \in \{10^5, 2 \cdot 10^5, 5 \cdot 10^5, 10^6, 2 \cdot 10^6, 5 \cdot 10^6, 10^7\}.$$

- $\rho = n/d \in \{100, 50, 20, 10, 5, 2, 1\}$.
- Sweep parametri su $\rho \in \{1, 10, 100\}$.
- HLL++: $k \in \{8, 10, 12, 14, 16, 18\}$.
- HLL/LogLog: $k \in \{4, 6, 8, 10, 12, 14, 16\}$.
- PC: $L \in \{4, 8, 12, 16, 20, 24, 28, 31\}$.

Merge HLL++

- Omogeneo: $\rho \in \{1, 10, 100\}$, $k \in \{10, 14, 18\}$, quattro hash.
- Hash mismatch: $k = 14$, seed mismatch e family mismatch.
- Precision mismatch:
 $(10, 14), (14, 10), (10, 18), (18, 10), (14, 18), (18, 14)$.
- Strategie: direct, reject, reduce_then_merge, unsafe_naive_merge.

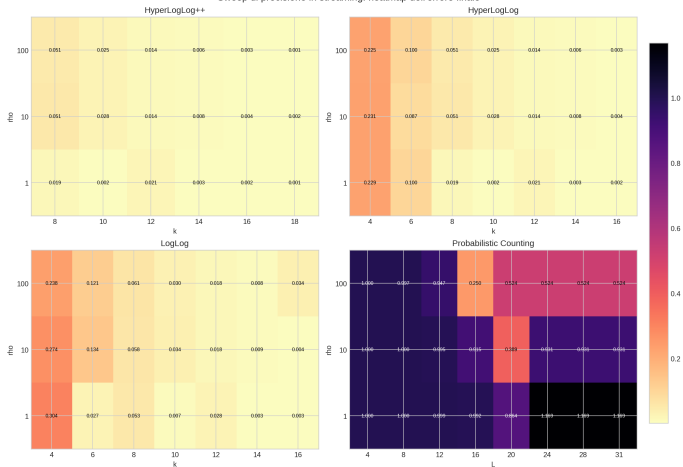
Risultato 1: comportamento degli algoritmi in streaming



- Su $F_0(t)$, HLL e HLL++ sono quasi sovrapposti: errore medio circa 0.00618.
- Su ρ_t , HLL++ è il migliore: $3.95 \cdot 10^{-4}$, contro $2.08 \cdot 10^{-3}$ di HLL.
- LogLog è più sensibile e nella seconda vista raggiunge circa 1.455.
- Probabilistic Counting resta instabile: circa 0.770 nella prima vista e 0.346 nella seconda.

Risultato 2: l'aumento di precisione favorisce soprattutto HLL++

Sweep di precisione in streaming: heatmap dell'errore finale



Miglior parametro osservato

- HLL++: $k = 18$ per tutti i valori di ρ .
 - HLL: $k = 16$ in media; con $\rho = 1$, $k = 10$.
 - LogLog: $k = 14$ in media; con $\rho = 10$, $k = 16$.
 - PC: $L = 20$ in media; con $\rho = 100$, $L = 16$.
-
- HLL++ è il metodo con andamento più regolare e prevedibile.
 - Gli altri metodi dipendono di più dal parametro.

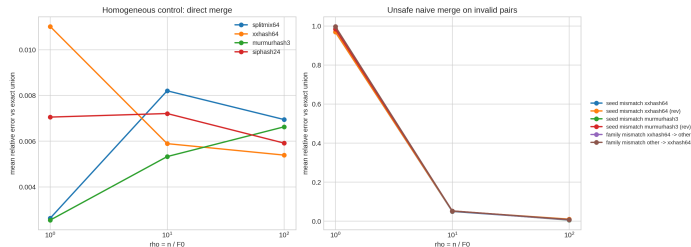
Risultato 3: il merge omogeneo è il controllo del caso valid

- Configurazione: HyperLogLog++, $\rho \in \{1, 10, 100\}$, $k \in \{10, 14, 18\}$, quattro funzioni di hash.
- Condizione di omogeneità: stessa funzione di hash, stesso seed e stessi parametri sui due lati.
- Totale osservato: 36 configurazioni e 900 coppie di merge.

$$\Delta_{abs} = 0, \quad \Delta_{rel} = 0$$

ρ	d	Coppie totali	Coppie con delta $\neq 0$	Max Δ_{rel}
1	10^7	300	0	0
10	10^6	300	0	0
100	10^5	300	0	0

Risultato 4: hash mismatch = caso invalid

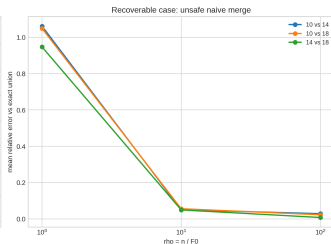
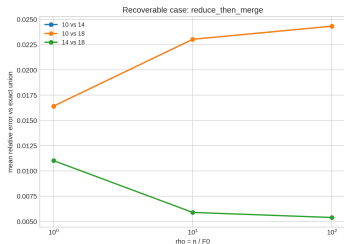


- Seed mismatch: stessa hash xxhash64 o murmurhash3, ma seed diversi.
- Family mismatch: xxhash64 contro splitmix64, murmurhash3 o siphash24; anche direzione inversa.
- Controllo omogeneo locale: errore medio = 0.006233.
- Casi invalid: errore medio tra 0.342374 e 0.352802.
- Errore massimo osservato: 1.009634.
- Strategia corretta: reject; il merge forzato è solo controllo negativo.

Conclusione

Se cambia la semantica dell'hash, il merge forzato perde significato. In questo contesto il caso va rifiutato.

Risultato 5: precision mismatch = caso recoverable



- Hash xxhash64, stesso seed.
- Coppie: (10, 14), (10, 18), (14, 18) e versioni invertite.
- reject: nessuna stima di merge, riferimento seriale = 0.016651.
- La normalizzazione avviene alla precisione minima comune.

Strategia	$\bar{\epsilon}_{merge}$
unsafe_naive_merge	0.363882
reduce_then_merge	0.016651

- reduce_then_merge coincide con il riferimento corretto.
- Il merge forzato raggiunge massimo 1.061378 e degrado medio 32.37.

Sintesi dei risultati e contributi

Risultati empirici

- HLL++ è il metodo più stabile nel dominio osservato.
- Il merge omogeneo coincide col riferimento seriale.
- L'eterogeneità dell'hash rompe la correttezza del merge.
- Il mismatch di precisione è recuperabile solo tramite normalizzazione esplicita.

Contributi del lavoro

- Implementazione uniforme di più algoritmi count-distinct.
- Interfaccia comune per hash, dataset, metriche e serializzazione.
- Framework riutilizzabile per streaming e merge distribuito.
- Lessico operativo `valid/recoverable/invalid`.

Take-away

Negli sketch count-distinct distribuiti, hash e parametri fanno parte della semantica dello sketch: senza compatibilità semantica il merge non è un'operazione neutra.

Limiti del lavoro

- Dataset soprattutto sintetico e controllato
- Studio del merge concentrato soprattutto su casi pairwise
- Analisi eterogenea limitata a hash mismatch e precision mismatch
- Parte di merge sviluppata soprattutto su HLL++

Direzioni future

- Dati reali e integrazione con sorgenti come Apache Kafka
- Topologie distribuite più ampie e scelta del punto ottimale di riduzione alla precisione minima
- Altri assi di eterogeneità semanticamente chiari
- Integrazione di sketch di cardinalità più recenti
- Misura del costo di normalizzazione e del costo cumulativo del merge

Grazie per l'attenzione.